Cremería Santa Fe

App híbrida Java + React · v2.0

Arquitectura y Pantallas

Contenido

1. Arquitectura
2. Pantallas

Parte 1 · Arquitectura

Este documento explica cómo se conectan las piezas de la app: qué hace cada capa, cómo viaja una petición desde un botón de React hasta SQLite y qué reglas de seguridad se aplican.

---

1. Visión general: app híbrida

Una app **híbrida** mezcla código nativo con una interfaz web. Aquí:

Capa	Tecnología	Responsabilidad
Interfaz	React 18 + Vite	Pantallas, formularios, navegación, modales. No guarda datos.
Puente	`WebView` + `@JavascriptInterface`	Permite que JavaScript llame a métodos de Java.
Lógica y datos	Java + SQLite	Sesión, permisos, validaciones, ventas, usuarios, notificaciones.

La regla de oro del proyecto: **la interfaz propone, Java decide.** React puede validar para dar una buena experiencia, pero la validación que cuenta está en Java.

---

2. Archivos Java (`android/app/src/main/java/com/cremeria/santafe/`)

`MainActivity.java`

Única `Activity` de la app. Al abrir:

1. Crea el canal de notificaciones (`Notificador.crearCanal`).
2. Pide el permiso `POST_NOTIFICATIONS` en Android 13+.
3. Crea un `WebView` y lo pone como contenido de la pantalla.
4. Configura el `WebView` (JavaScript, DOM storage y acceso a archivos locales; ver §7).
5. Registra el puente: `webView.addJavascriptInterface(new WebAppInterface(this), "Android")`. Por eso en React existe `window.Android`.
6. Carga `file:///android_asset/www/index.html` (la app React compilada).

El botón *atrás* navega hacia atrás dentro del `WebView` si puede; si no, cierra la app. La orientación está fija en vertical (`AndroidManifest.xml`).

`WebAppInterface.java`

Es **el puente**. Cada método con `@JavascriptInterface` puede llamarse desde React. Responsabilidades:

- Mantener la **sesión** (`rolActual`, `idActual`) en memoria.
- Bloquear el login 30 s tras 5 intentos fallidos.
- Validar **permisos por rol** (`haySesion()`, `soloAdmin()`) antes de ejecutar cada acción.
- Validar datos de entrada (precio > 0, unidad válida, formato de usuario, etc.).
- Guardar los **ajustes** (número de WhatsApp y notificaciones) en `SharedPreferences`.
- Armar el mensaje y el `Intent` de WhatsApp (`https://wa.me/<numero>?text=...`).

Todos los métodos devuelven un **String JSON**:

```
{ "ok": true,  "data": ... }
{ "ok": false, "error": "Mensaje listo para mostrar al usuario" }
```

`DatabaseHelper.java`

Extiende `SQLiteOpenHelper`. Crea las tablas, ejecuta las consultas y contiene la lógica transaccional de ventas y entradas. También guarda y verifica contraseñas (salt + SHA-256).

`Categorias.java`

Catálogo de categorías y reglas por palabra clave. Detecta a qué categoría pertenece un nombre (normaliza acentos y mayúsculas, reconoce plural `s` / `es`) y rechaza guardados inconsistentes.

`Notificador.java`

Crea el canal `stock_bajo` y muestra una notificación `InboxStyle` con los productos que quedaron en su mínimo. Incluye un botón **Avisar por WhatsApp**. Usa un ID fijo (`1001`): una notificación nueva reemplaza a la anterior.

---

3. Base de datos (SQLite)

Archivo: `cremeria_santa_fe.db` · Versión: **2** (`DB_VERSION`).

`productos`

Columna	Tipo	Notas
`id`	INTEGER	PK, autoincremental
`nombre`	TEXT	Obligatorio
`categoria`	TEXT	Por defecto `'Otros'`
`precio`	REAL	Precio por unidad
`unidad`	TEXT	`Kg`, `Lt` o `Pza` (por defecto `'Kg'`)
`cantidad`	REAL	Existencia actual
`umbralMinimo`	REAL	Stock mínimo (por defecto 5)
`vecesVendido`	REAL	Acumulado vendido (base del reporte)

`ventas` y **`venta_items`**

Tabla	Columnas
`ventas`	`id` (PK), `fecha` (INTEGER, milisegundos), `total` (REAL)
`venta_items`	`id` (PK), `ventaId` (→ `ventas.id`), `productoId` (→ `productos.id`), `productoNombre` (TEXT), `cantidad` (REAL), `subtotal` (REAL)

`entradas` y **`usuarios`**

Tabla	Columnas
`entradas`	`id` (PK), `productoId` (→ `productos.id`), `productoNombre` (TEXT), `cantidad` (REAL), `proveedor` (TEXT), `fecha` (INTEGER, milisegundos)
`usuarios`	`id` (PK), `usuario` (TEXT, único, sin distinguir mayúsculas), `clave` (TEXT, formato `salt:sha256(salt+clave)`), `rol` (`'admin'` o `'empleado'`)

Notas:

- Las relaciones (`ventaId`, `productoId`) **no** están declaradas como `FOREIGN KEY`; se mantienen desde el código. `venta_items` y `entradas` guardan también `productoNombre` para conservar el nombre aunque el producto cambie.
- `cantidad` y `vecesVendido` son `REAL` porque se vende por kilo/litro (decimales). Las operaciones usan `ROUND(..., 3)` para evitar errores de coma flotante.
- **Migración v1 → v2:** `onUpgrade` solo agrega la tabla `usuarios` (y siembra los usuarios iniciales). No se borra ningún dato.
- Los comentarios `RF01…RF08` en el código corresponden a los **requisitos funcionales** del proyecto (productos, alertas,

ventas, entradas, etc.).

---

4. Ciclo de una llamada (ejemplo: confirmar una venta)

```
SalesRegistration.jsx
  └ confirmarVenta()
      └ bridge.registrarVenta(items)                    (bridge.js)
          └ window.Android.registrarVenta(JSON.stringify(items))
              └ WebAppInterface.registrarVenta(String)       ← Java
                  ├ ¿hay sesión?                             si no → {ok:false}
                  └ DatabaseHelper.registrarVenta(JSONArray)
                      ├ une líneas repetidas del mismo producto
                      ├ BEGIN TRANSACTION
                      │   ├ valida existencia (no vende más de lo que hay)
                      │   ├ calcula total con precios DE LA BASE DE DATOS
                      │   ├ INSERT ventas + venta_items
                      │   ├ UPDATE productos (cantidad, vecesVendido)
                      │   └ detecta alertas (cruzó el mínimo o se agotó)
                      └ COMMIT   (si algo falla, se revierte todo)
                  ├ si hay alertas y notificaciones activas → Notificador
                  └ return {"ok":true,"data":{ventaId,total,alertas[]}}
          └ bridge.js: parsea el JSON; si ok=false lanza Error(mensaje)
      └ la pantalla muestra ModalExito, ModalStockBajo o ModalError
```

Detalles importantes:

- **No se confía en el precio que manda la pantalla.** Java lee el precio de la base de datos.
- Una alerta de stock se dispara solo cuando la venta **cruza** el mínimo (`antes > umbral && después <= umbral`) o **agota** el producto. Así no se repite la alerta en cada venta de un producto que ya estaba bajo.
- Todo es una **transacción**: o se registra la venta completa o no se registra nada.

---

5. El puente en detalle (`window.Android`)

`react-app/src/bridge.js` envuelve `window.Android`:

- Si `window.Android` existe (dentro de la app), llama a Java y convierte la respuesta JSON: `ok:true` → devuelve `data`; `ok:false` → lanza `Error(error)`.
- Si **no** existe (navegador con `npm run dev`), usa un **simulador** en memoria que imita las mismas reglas para poder desarrollar la interfaz.

Así las pantallas siempre usan el mismo código:

```
try {
  const productos = await bridge.obtenerProductos();
} catch (e) {
  setError(e.message);   // → ModalError
}
```

Métodos disponibles

Método	Parámetros	Devuelve (`data`)	Permiso
`login`	`usuario, clave`	`{id, usuario, rol}`	Todos
`cerrarSesion`	—	`true`	Todos

`obtenerCategorias`	—	`["Lácteos y Quesos", …]`	Sesión
`categoriaSugerida`	`nombre`	`"Categoría"` o `""`	Sesión
`obtenerProductos`	—	`[producto]`	Sesión
`registrarVenta`	`itemsJson` (`[{productoId, cantidad}]`)	`{ventaId, total, alertas[]}`	Sesión
`avisarStockBajoWhatsApp`	—	`true` (abre WhatsApp)	Sesión
`guardarProducto`	`nombre, categoria, precio, unidad, cantidad, umbral`	`{id}`	Admin
`registrarEntrada`	`productoId, cantidad, proveedor`	`{id}`	Admin
`obtenerProductosFaltantes`	—	`[producto]` con `cantidad <= umbralMinimo`	Admin
`obtenerMasVendidos`	`limite`	`[producto]` por `vecesVendido`	Admin
`obtenerResumenDia`	—	`{productosActivos, alertasStock, ventasHoy}`	Admin
`obtenerAjustes`	—	`{whatsappNumero, notificaciones, claveAdminPorDefecto}`	Admin
`guardarAjustes`	`whatsappNumero, notificaciones`	`true`	Admin
`listarUsuarios`	—	`[{id, usuario, rol}]`	Admin
`crearUsuario`	`usuario, clave, rol`	`true`	Admin
`eliminarUsuario`	`id`	`true`	Admin
`cambiarContrasena`	`id, nueva`	`true`	Admin

> `bridge.registrarEntrada(productoId, _nombre, cantidad, proveedor)` conserva el parámetro `_nombre` por compatibilidad con la pantalla, pero ya **no se envía a Java**: el nombre se toma de la base de datos.

---

6. Reglas de negocio

Roles y permisos

- Se validan en `WebAppInterface` con `haySesion()` y `soloAdmin()`.
- En React, `App.jsx` fuerza a un empleado a ver siempre *Ventas*. Aunque alguien manipulara la interfaz, Java rechazaría cualquier acción de administrador con "*No tienes permiso...*".
- Reglas de usuarios: no puedes eliminarte a ti mismo y siempre debe quedar al menos un admin.

Categorías

- Definidas en el bloque `static` de `Categorias.java` (`LinkedHashMap<categoría, palabras>`).
- `sugeridas(nombre)` devuelve las categorías que coinciden; `validar(nombre, categoria)` rechaza si hay coincidencias y la elegida no está entre ellas. Si el nombre no coincide con nada, se acepta cualquier categoría.
- La categoría **Otros** no tiene palabras clave y acepta cualquier producto.
- El listado de ejemplo en `bridge.js` solo sirve para el simulador; **la regla real es la de Java**.
- Para agregar una categoría: añade una línea `REGLAS.put("Nombre", new String[]{...})`. Las palabras van en minúsculas, sin acentos y en singular.

Stock bajo

- Cada producto tiene su propio `umbralMinimo` (por defecto 5).
- Al vender: si el producto cruza su mínimo o se agota → entra en `alertas[]`.
- Con `alertas` y las notificaciones activadas (`SharedPreferences`, clave `notificaciones`), se lanza la notificación del sistema.

- `avisarStockBajoWhatsApp` toma **todos** los productos con `cantidad <= umbralMinimo`.

WhatsApp

- El número se guarda solo con dígitos. Si son 10 dígitos se antepone `52` (México).
- Se valida una longitud de 11 a 15 dígitos.
- Se abre `https://wa.me/<numero>?text=<mensaje codificado>` con `Intent.ACTION_VIEW`.

---

7. Seguridad

Tema	Cómo está resuelto	Observación
Contraseñas	`salt` aleatorio de 16 bytes + SHA-256, guardado como `salt:hash`. Comparación con `MessageDigest.isEqual`.	Una sola pasada de hash; para más resistencia usar PBKDF2/bcrypt/Argon2.
Fuerza bruta	5 intentos fallidos → bloqueo de 30 s.	El contador vive en memoria (se reinicia al cerrar la app).
Permisos	Validados en Java por rol.	—
Inyección SQL	Consultas con parámetros (`?`).	—
Precios	Se leen de la base, no de la pantalla.	—
Depuración del WebView	`setWebContentsDebuggingEnabled` solo si la app es *debuggable*.	—
Acceso a archivos del WebView	`AllowFileAccessFromFileURLs` y `AllowUniversalAccessFromFileURLs` habilitados.	Necesario para módulos Vite bajo `file://`. No cargar contenido externo en este `WebView`.
Datos en reposo	SQLite sin cifrar; `allowBackup="true"`.	Considerar SQLCipher o `allowBackup="false"` en producción.
Credenciales iniciales	`admin / admin123` y `empleado / empleado123`.	La app avisa mientras el admin conserve la clave por defecto.

---

8. Compilación y empaquetado de la interfaz

- `vite.config.js` usa `base: './'` para que las rutas de los recursos sean **relativas** y funcionen desde `file:///android_asset/www/index.html`.
- `npm run build` genera `react-app/dist/` (`index.html` + `assets/index-<hash>.js|css`).
- Ese contenido se copia a `android/app/src/main/assets/www/`; Gradle lo empaqueta dentro del APK.
- Como el nombre del archivo lleva un hash, conviene **borrar `www` antes de copiar** para no acumular versiones viejas.

---

9. Configuración de Android

Elemento	Valor
`applicationId` / `namespace`	`com.cremeria.santafe`
`compileSdk` / `targetSdk`	34
`minSdk`	24 (Android 7.0)
Java	17
Android Gradle Plugin	8.5.2
Dependencias	`androidx.appcompat:appcompat:1.7.0`, `androidx.webkit:webkit:1.11.0`,

	`com.google.android.material:material:1.12.0`
Permisos	`POST_NOTIFICATIONS` (Android 13+)
Actividad	`MainActivity` , vertical, `windowSoftInputMode="adjustResize"`

Parte 2 · Pantallas

¿Dónde se construyeron las pantallas?

Las pantallas **no son archivos XML de Android**. Se construyeron con **React** y viven en:

```
react-app/src/screens/      ← una pantalla por archivo .jsx
react-app/src/components/   ← piezas compartidas (barra inferior y modales)
react-app/src/styles.css    ← estilos de toda la app
```

El camino que sigue lo que ves en el teléfono:

```
1. Escribes/editas  react-app/src/screens/*.jsx
2. npm run build   → react-app/dist/   (index.html + JS + CSS compilados)
3. Copias dist/*   → android/app/src/main/assets/www/
4. MainActivity.java abre un WebView con file:///android_asset/www/index.html
5. El WebView ejecuta React y dibuja las pantallas
```

Por eso, si abres `android/` en Android Studio, **no encontrarás layouts** (`res/layout/`): solo hay una `MainActivity` con un `WebView`. Todo lo visual está en `react-app/`.

> Otra pista: en `android/app/src/main/assets/www/assets/` hay un `index-XXXX.js` ilegible. Es el resultado compilado de todas las pantallas juntas. **No se edita a mano**; se regenera con `npm run build`.

Cómo funciona la navegación

No se usa `react-router`. La navegación es un `switch` en `react-app/src/App.jsx` controlado por dos estados:

```
        const [sesion, setSesion]     = useState(null);        // { id, usuario, rol }
        const [pantalla, setPantalla] = useState('dashboard'); // pantalla actual
```

1. Si **no hay sesión** → se muestra `Login`.
2. Al iniciar sesión: el **admin** va a `dashboard`; el **empleado** va a `ventas`.
3. Si hay sesión, un `switch (destino)` decide qué pantalla dibujar.
4. Cada pantalla recibe `onNavegar('nombreDePantalla')` (o `onVolver`) para cambiar de pantalla.
5. Un empleado siempre ve `ventas`, sin importar el valor de `pantalla`.

Mapa de pantallas

```
Login
├── admin ──► Dashboard
│             ├── Productos ──► Nuevo producto
│             ├── Inventario ──► Entrada de mercancía
│             ├── Ventas
│             ├── Reportes (Productos más vendidos)
│             └── Ajustes
│
└── empleado ──► Ventas   (es la única pantalla que puede ver)
```

Claves de ruta usadas en `onNavegar`:

Clave	Pantalla
`dashboard`	`Dashboard.jsx`
`productos`	`ProductList.jsx`
`nuevoProducto`	`ProductRegistration.jsx`
`inventario`	`InventoryControl.jsx`
`entradaMercancia`	`MerchandiseEntry.jsx`
`ventas`	`SalesRegistration.jsx`
`topProductos`	`TopProducts.jsx`
`ajustes`	`Settings.jsx`

Detalle de cada pantalla

1. Login — `screens/Login.jsx`

- **Qué muestra:** logo, formulario de usuario y contraseña, versión.
- **Puente:** `login(usuario, clave)`.
- **Comportamiento:** si falla, abre `ModalError` y limpia la contraseña. Si funciona, avisa a `App` con `onIngresar(sesion)`.
- **Acceso:** todos.

2. Dashboard — `screens/Dashboard.jsx`

- **Qué muestra:** saludo con la fecha, cuatro accesos rápidos (Productos, Inventario, Ventas, Reportes), tarjeta de alerta si hay stock bajo, **Resumen del Día** (productos activos, alertas y ventas de hoy) y acceso a Ajustes.
- **Puente:** `obtenerResumenDia`, `obtenerAjustes` (para saber si el admin aún usa `admin123`), `avisarStockBajoWhatsApp`.
- **Detalles:** botón *Salir* (cierra sesión). Muestra un aviso naranja si la contraseña del admin sigue siendo la de fábrica.
- **Acceso:** solo admin.

3. Productos — `screens/ProductList.jsx`

- **Qué muestra:** buscador, chips de categorías (`Todos` + las de Java), lista de productos con precio, categoría y existencia (punto verde o naranja según el stock mínimo).
- **Puente:** `obtenerProductos`, `obtenerCategorias`.
- **Detalles:** botón flotante **+** que lleva a `nuevoProducto`. El filtrado y la búsqueda se hacen en React sobre la lista ya cargada.
- **Acceso:** solo admin (el empleado no llega a esta pantalla).

4. Nuevo Producto — `screens/ProductRegistration.jsx`

- **Qué muestra:** formulario con nombre, precio, cantidad + unidad (`Kg` / `Lt` / `Pza`), stock mínimo (5 por defecto) y categoría.
- **Puente:** `obtenerCategorias`, `categoriaSugerida`, `guardarProducto`.
- **Detalles:** al escribir el nombre se llama a `categoriaSugerida` y la categoría se **selecciona sola**. Si el usuario elige otra, aparece un aviso. Java hace la validación final y puede rechazar el guardado (categoría incorrecta, nombre duplicado, datos inválidos).
- **Acceso:** solo admin.

5. Inventario — `screens/InventoryControl.jsx`

- **Qué muestra:** contador de productos críticos, botón de aviso por WhatsApp, lista **ordenada por urgencia** con insignia y barra de progreso (naranja = bajo, rojo = sin stock) y hasta tres productos saludables como referencia.
- **Puente:** `obtenerProductos`, `obtenerProductosFaltantes`, `avisarStockBajoWhatsApp`.
- **Detalles:** botón flotante **+** que lleva a `entradaMercancia`.
- **Acceso:** solo admin.

6. Entrada de Mercancía — `screens/MerchandiseEntry.jsx`

- **Qué muestra:** selector de producto, cantidad recibida (con la unidad del producto elegido), proveedor y fecha del día (solo lectura).
- **Puente:** `obtenerProductos`, `registrarEntrada`.
- **Detalles:** suma la cantidad al inventario y guarda la entrada. Al aceptar, regresa a Inventario.
- **Acceso:** solo admin.

7. Nueva Venta — `screens/SalesRegistration.jsx`

- **Qué muestra:** una o más **líneas de venta** (producto + cantidad con botones − / +), el disponible y el precio por unidad, subtotal y total estimado, botón para agregar otro producto y **Confirmar Venta**.
- **Puente:** `obtenerProductos`, `registrarVenta`, `avisarStockBajoWhatsApp`.
- **Detalles:**
  - Los productos agotados aparecen deshabilitados en el selector.
  - El paso de los botones es de 0.5 para `Kg` / `Lt` y de 1 para `Pza`.
  - Si la cantidad excede la existencia se avisa en pantalla, y Java vuelve a validar.
  - Resultado: `ModalExito`, o `ModalStockBajo` si la venta dejó productos en el mínimo.
  - El **empleado** ve esta pantalla sin barra inferior, con botón *Salir* y su nombre de sesión. El admin la ve con la barra inferior.
- **Acceso:** admin y empleado.

8. Productos Más Vendidos — `screens/TopProducts.jsx`

- **Qué muestra:** pestañas Hoy / Semana / Mes y un ranking con barras proporcionales al producto más vendido.
- **Puente:** `obtenerMasVendidos(8)`.
- **Nota:** las pestañas son solo visuales; el ranking usa el acumulado histórico (`vecesVendido`).
- **Acceso:** solo admin (desde *Reportes* en el Dashboard).

9. Ajustes — `screens/Settings.jsx`

- **Qué muestra:** aviso si el admin usa la clave por defecto, **Alertas de stock bajo** (WhatsApp del dueño e interruptor de notificaciones) y **Usuarios** (lista, cambiar contraseña, eliminar y agregar).
- **Puente:** `obtenerAjustes`, `guardarAjustes`, `listarUsuarios`, `crearUsuario`, `eliminarUsuario`, `cambiarContrasena`.
- **Detalles:** la acción de eliminar pide confirmación con `ModalConfirmar`. No se puede eliminar al usuario con sesión activa.
- **Acceso:** solo admin.

Componentes compartidos (`react-app/src/components/`)

`BottomNav.jsx`

Barra inferior con cuatro botones: **Inicio, Productos, Inventario, Ventas**. Recibe `activo` (para resaltar el actual) y `onNavegar`. Los íconos son SVG dibujados en el propio componente. Se muestra en Dashboard, Productos, Inventario y Ventas (cuando es admin).

`Modal.jsx`

Cuatro modales reutilizables:

Componente	Uso
`ModalError`	Muestra errores devueltos por Java (`e.message`).
`ModalExito`	Confirma una operación correcta.
`ModalStockBajo`	Lista los productos que se están acabando tras una venta y ofrece avisar por WhatsApp.

`ModalConfirmar`	Sustituye a `window.confirm()`, que **no funciona dentro del WebView**.

Estilos (`react-app/src/styles.css`)

- Los colores son variables CSS en `:root` (`--purple`, `--orange`, `--green`, `--red`, `--bg` ...). Cambiar la paleta de la app es editar ese bloque.
- Clases principales: `.pantalla` (contenedor, máximo 480 px), `.encabezado`, `.contenido`, `.tarjeta`, `.campo`, `.boton-primario`, `.boton-secundario`, `.chip`, `.insignia`, `.barra-progreso`, `.modal`, `.barra-navegacion`.
- Las pantallas también usan estilos en línea (`style={{…}}`) para ajustes puntuales.

Patrón que repiten las pantallas

```
export default function MiPantalla({ onNavegar }) {
  const [datos, setDatos] = useState([]);
  const [error, setError] = useState(null);

  // 1) Cargar datos de Java al abrir la pantalla
  useEffect(() => {
    bridge.obtenerProductos().then(setDatos).catch((e) => setError(e.message));
  }, []);

  // 2) Acciones del usuario → llamar al puente
  const guardar = async () => {
    try { await bridge.guardarProducto(/* ... */); }
    catch (e) { setError(e.message); }   // 3) Errores → ModalError
  };

  return (
    <div className="pantalla">
      {/* encabezado, contenido, BottomNav, modales */}
    </div>
  );
}
```

Cómo agregar una pantalla nueva

1. Crea `react-app/src/screens/MiPantalla.jsx` siguiendo el patrón anterior.
2. En `App.jsx`: importa el componente y agrega un `case 'miPantalla':` en el `switch`.
3. Desde otra pantalla, navega con `onNavegar('miPantalla')`.
4. Si necesita datos nuevos, agrega el método en `WebAppInterface.java` (con `@JavascriptInterface`, validando sesión/permisos), la lógica en `DatabaseHelper.java`, y declara el método en `bridge.js` (lista `directos` y simulador `mock`).
5. `npm run build`, copia `dist/*` a `android/app/src/main/assets/www/` y ejecuta la app.